This is an open access article published under a Creative Commons Attribution (CC-BY) License, which permits unrestricted use, distribution and reproduction in any medium, provided the author and source are cited.

ACS AUTHORCHOICE

ACS Synthetic Biology

pubs.acs.org/synthbio

Research Article

Mutation Maker, An Open Source Oligo Design Platform for Protein Engineering

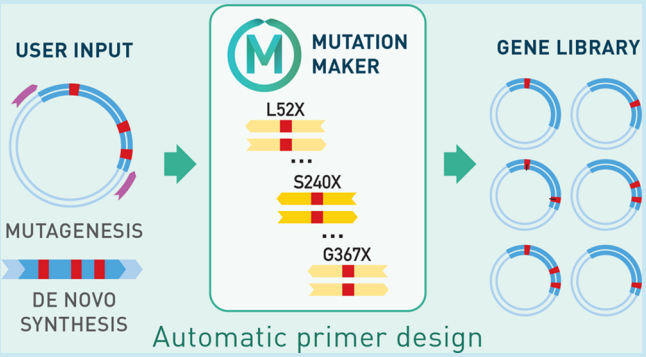
Kaori Hiraga,[⊥] Petr Mejzlik,[⊥] Matej Marcisin, Nikita Vostrosablin, Anna Gromek, Jakub Arnold, Sebastian Wiewiora, Rastislav Svarba, David Prihoda, Kamila Clarova, Ondrej Klempir, Josef Navratil, Ondrej Tupa, Alejandro Vazquez-Otero, Marcin W. Walas, Lukas Holy, Martin Spale, Jakub Kotowski, David Dzamba, Gergely Temesi, Jay H. Russell, Nicholas M. Marshall, Grant S. Murphy, and Danny A. Bitton*

Cite This: *ACS Synth. Biol.* 2021, 10, 357−370

Read Online

ACCESS | Metrics & More | Article Recommendations | Supporting Information

ABSTRACT: Protein engineering is the discipline of developing useful proteins for applications in research, therapeutic, and industrial processes by modification of naturally occurring proteins or by invention of *de novo* proteins. Modern protein engineering relies on the ability to rapidly generate and screen diverse libraries of mutant proteins. However, design of mutant libraries is typically hampered by scale and complexity, necessitating development of advanced automation and optimization tools that can improve efficiency and accuracy. At present, automated library design tools are functionally limited or not freely available. To address these issues, we developed Mutation Maker, an open source mutagenic oligo design software for large-scale protein engineering experiments. Mutation Maker is not only specifically tailored to multisite random and directed mutagenesis protocols, but also pioneers bespoke mutagenic oligo design for *de novo* gene synthesis workflows. Enabled by a novel bundle of orchestrated heuristics, optimization, constraint-satisfaction and backtracking algorithms, Mutation Maker offers a versatile toolbox for gene diversification design at industrial scale. Supported by *in silico* simulations and compelling experimental validation data, Mutation Maker oligos produce diverse gene libraries at high success rates irrespective of genes or vectors used. Finally, Mutation Maker was created as an extensible platform on the notion that directed evolution techniques will continue to evolve and revolutionize current and future-oriented applications.



KEYWORDS: *synthetic biology, protein engineering, protein design, directed evolution, site-scanning saturation mutagenesis, multi site-directed mutagenesis, PCR-based accurate synthesis, gene synthesis*

Downloaded via COLD SPRING HARBOR LABORATORY on April 5, 2026 at 16:27:53 (UTC).
See https://pubs.acs.org/sharingguidelines for options on how to legitimately share published articles.

The process of natural selection has generated a large set of protein sequences with a broad array of biologically important functions (*e.g.*, binding, scaffolding, and catalysis). While the set of extant protein sequences is large (>10^6), it is only a miniscule fraction of all possible protein sequences.[1] Indeed, early efforts in protein design and engineering established that natural protein sequences could be modified to possess properties useful outside biological systems and that entirely *de novo* proteins could be created with structures and functions not observed in nature.[2−7] Further advances in directed evolution and protein design combined with decreasing costs of DNA synthesis and sequencing have led to a new wave of biological engineering referred to as synthetic biology. Synthetic biology seeks to redesign naturally occurring biological system (*e.g.*, enhancing metabolic pathways to produce natural products) or to engineer *de novo* biological systems (*e.g.*, invention of entirely novel metabolic pathways).[8]

Central to the efforts of synthetic biology is the ability to rapidly assemble novel genes and introduce mutations as part of a large-scale screen and/or selection. Synthetic biology efforts to engineer metabolic pathways to produce industrially or pharmaceutically important compounds typically require numerous iterative rounds of gene assembly, mutagenesis, and screening. Given the significant resources required to undertake these efforts, new methods for gene assembly, mutagenesis, and screening are of considerable interest.

Received: October 23, 2020
Published: January 12, 2021

ACS Publications

© 2021 The Authors. Published by American Chemical Society

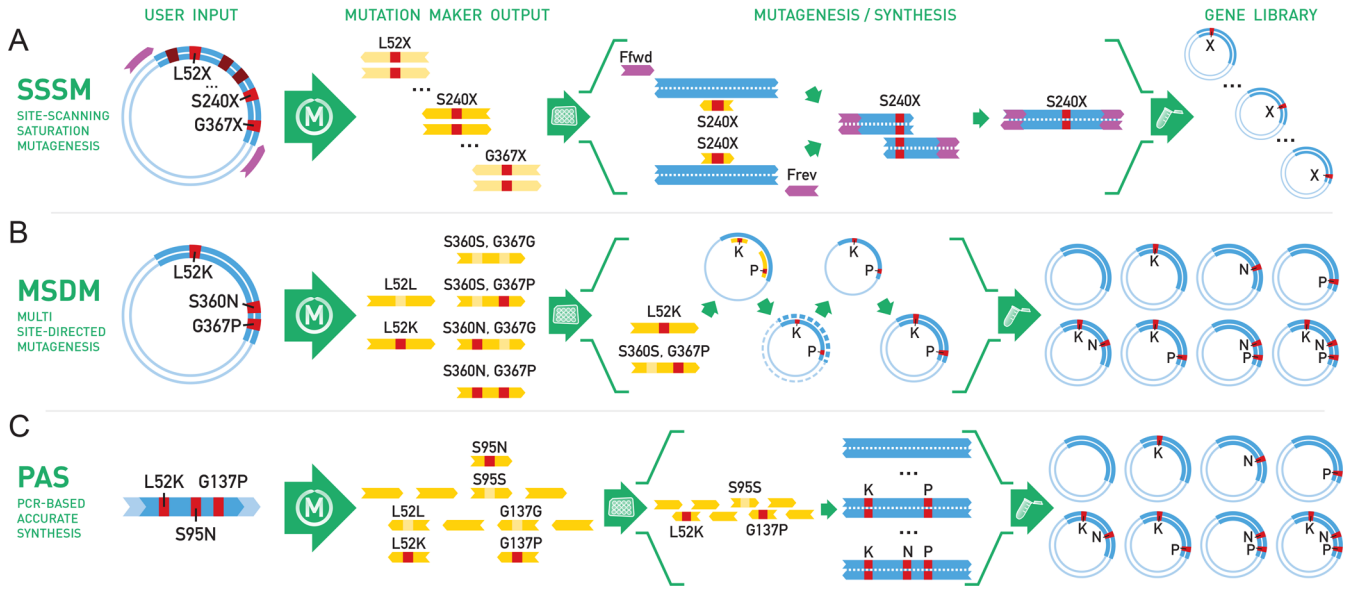


Figure 1. Schematic representations of site-scanning saturation mutagenesis (SSSM), multi site directed mutagenesis (MSDM), and gene synthesis by PCR-based accurate synthesis (PAS) workflows. (A, left to right) SSSM workflow: User's input includes plasmid sequence (blue) with gene of interest, a list of amino acid positions to be changed (red squares, only 3 sites are labeled for simplicity), forward and reverse flanking primers (Ffwd and Frev, purple arrows), degenerate codon (marked as "X"), and a list of desired parameters (T_m, GC content, flanking primers, *etc.*, not shown). Mutation Maker outputs pairs of forward and reverse overlapping mutagenic primers (yellow) for multiple parallel PCR reactions in multiple 96-well plates (green arrow). To create N- and C- gene fragments, PCR reactions are first conducted separately and in parallel for each site (top and bottom rows, respectively) using mutagenic (yellow with red squares) and flanking primer pairs (purple) and a gene template (blue). The overlapping N- and C- fragments are then stitched together using flanking primers (purple) *via* a second PCR reaction (SOEing, Splicing by Overlap Extension) that produce full-length gene variants. These are subsequently pooled to create a gene library, where each variant carries a single random amino acid substitution at a specified position. Thereafter, the library is cloned into an expression vector (blue circles). (B, left to right) MSDM workflow: User input includes gene of interest and a list of target residues and mutations (red squares, only 3 mutations are shown for simplicity) and a list of parameters (T_m, GC, host organism, degeneracy, *etc.*, not shown). Mutation Maker outputs a set of unidirectional mutagenic primers (yellow with red squares) as well as primers carrying parental codons (light yellow squares on primers) that cover one or more target sites. The resultant primers can be used in MSDM experiments using the QuikChange lightning multi site-directed mutagenesis kit (Agilent) to yield diverse combinatorial library (blue circles). (C, left to right) *De novo* gene synthesis workflow using the PCR-based Accurate Synthesis (PAS) protocol. The input to mutation maker consists of gene of interest (blue line) with a list of target residues and mutations (red squares, as indicated) as well as and a list of constraints (T_m, GC, host organism, motifs to exclude, degeneracy, *etc.*, not shown). Mutation Maker outputs a set of overlapping oligos (yellow) that may carry mutation and parental codon combinations (red and light-yellow squares, respectively) in their nonoverlapping portions. These oligos can be assembled to full-length gene variants (blue lines), each carrying different mutation combinations that later can be cloned into an expression vector (blue circles).

Over the past 30 years, numerous techniques and commercial kits have been developed to assemble novel genes and systematically mutate genes at random or specific sites.[9−12] Site saturation mutagenesis (SSM) is a widely used technique for construction of a gene library in which a single residue is randomly substituted with all possible amino acids at a single or at multiple positions.[13−15] SSM can be combined with a "site-scanning" strategy[16] to form a technique we refer to as site-scanning saturation mutagenesis (SSSM, Figure 1A). SSSM can generate a diverse gene library with a theoretical diversity that is appropriately scaled to the throughput of the screening method. Briefly, in SSSM, a set of amino acids in a given protein sequence are subjected to random mutagenesis (at a single site out of defined positions per clone). Target sites might be selected by rational design based on sequence, structural or experimental information, or alternatively through an exhaustive scan of the entire sequence. SSSM is typically performed using mutagenic oligonucleotides (oligos) often referred to as primers, which enable parallel and deep mutational scans that often result in the discovery of beneficial single mutations. These single mutations can then be combined by multi site-directed mutagenesis (MSDM),[17] which generates a combinatorial gene library that permits a comprehensive search for additive mutational effects (Figure 1B). Both methods rely on annealing of mutagenic primers to complementary DNA sequences of a template gene, where mismatches at designated positions facilitate codon substitutions. When multiple amino acid substitutions at the same site are being considered, degenerate primers (oligonucleotides synthesized with multiple nucleotides at a particular position) are often employed to reduce the number of primers to be synthesized and consequently to save time and cost. Once gene libraries are constructed, the genes are transformed into a host (mixed with cell free expression reagents) to enable proteins to be expressed and screened for function.

In addition to the conventional mutagenesis methods described above, *de novo* gene synthesis methodologies also offer rapid, accurate, and cost-effective approaches for assembling and modifying gene sequences.[18,19] Gene synthesis methods do not require a DNA template and typically rely on synthetic oligonucleotides with overlapping overhangs that can be assembled into a full-length gene by repeated PCR extensions (Figure 1C).

Irrespective of the underlying protocol, the success of mutagenesis and *de novo* gene synthesis experiments is largely dependent on the design quality of mutagenic oligos. To

https://dx.doi.org/10.1021/acssynbio.0c00542
ACS Synth. Biol. 2021, 10, 357−370

ensure efficient PCR amplification and consequently precise introduction of mutations, a lengthy list of factors must be accounted for during the design phase (*e.g.*, melting temperature (T_m), GC content, primer size, mutation positions, hairpins, primer-dimers, *etc.*). Several commercial and academic tools were developed to enable design of mutagenic oligos for mutagenesis[16,20−22] and gene synthesis experiments focused on a limited number of positions or genes.[23−30] These tools were not intended to support the scale and complexity of mutagenic oligo design as required by industrial synthetic biology groups. Moreover, none of the available tools offer an automated and customizable design of mutagenic oligos for the construction of combinatorial gene libraries by gene synthesis methodologies. In addition, some of the published tools are no longer being supported, not freely available for widespread commercial use, or can handle only a limited number of mutations, if any.

To fill this gap, we developed Mutation Maker, an open source mutagenic oligo design software for mutagenesis and mutagenic *de novo* gene synthesis experiments. Mutation Maker features unique functionalities that are powered by several novel algorithms, providing an accurate, rapid, high-throughput and user-friendly platform for designing large-scale protein engineering experiments. Experimental validation of Mutation Maker revealed high success rates of mutagenic oligos across different genes, vectors and workflows. Mutation Maker workflows were designed primarily to support SSSM using an overlap extension PCR protocol,[31] MSDM using the QCLM kit (QuikChange lightning multi site-directed mutagenesis),[17] and gene synthesis using the PAS technique (PCR-based accurate synthesis).[19] Oligo design parameters are user-adjustable throughout all gene diversification modalities to allow seamless adoption of alternative and future oligonucleotide-based mutagenesis methods. Additionally, these flexible user inputs render Mutation Maker applicable to both random and rational design strategies, addressing the growing demand for unbiased, focused libraries.[32] Finally, Mutation Maker is freely available and open source software, so that the greater scientific community can both utilize the tool without restriction, as well as continue to develop and broaden its scope.

■ RESULTS

SSSM: Mutation Maker Offers Brute-Force or Fast-Approximation Mutagenic Oligo Designs. In principle, SSSM can reveal the functional consequences of any amino acid change at every position along a protein sequence.[12,16] SSSM employs multiple parallel PCR reactions that utilize overlapping mutagenic primer pairs and a SOEing PCR technique (splicing by overlap extension)[31] to introduce random mutations at specified locations (Figure 1A). To maximize the success of the numerous parallel PCR reactions that are typically performed using multiple 96-well plates, we developed two constraint-satisfaction algorithms, as follows. First, we developed a brute-force algorithm by which all potential primers around all mutation sites are precomputed, and thereafter systematically filtered to satisfy various constraints. Second, we built a fast-approximation algorithm by which mutagenic primers are dynamically extended until they satisfy the design criteria.

The minimal input for both SSSM algorithms is the plasmid sequence containing the parental gene of interest (GOI), the forward and reverse flanking primers and a list of mutation sites with their respective degenerate codon (default, "NNK"). Degenerate codons can be directly specified by the user or alternatively can be dynamically generated by a heuristic algorithm from a user-specified lists of amino acids, as described later.

To mitigate known primer-related issues such as primer self-annealing, primer-dimers, and low amplification efficiency, users can adjust the following primer parameters: hairpins and primer-dimers check, GC content, primer length, 5′ end length, 3′ end length, 3′ T_m range, and the overlapping segment size and its T_m range. Since T_m consistency across all parallel reactions (on each 96-well plate) is desired and essential to consistent high-quality PCR, users can also set the maximum T_m difference allowed across the entire set of primers and overlaps.

Both SSSM algorithms consider the lengths of the primer and its 3′ end as hard constraints that must be satisfied, while all remaining parameters are considered soft and may be violated. Mutation Maker also incorporates a scoring function with user-adjustable weights, which governs the logic of constraints violation and consequently of primer ranking. Once designed, the program displays the primers and their respective scores and notifies the user of the violated constraints.

In summary, Mutation Maker factors in multiple user-specified constraints to ensure flexible and reliable mutagenic primer design. It also provides a choice between a fast-approximation search and an exhaustive scan, which in essence reflects a trade-off between a shorter computation time and a higher probability of finding a better solution, respectively.

SSSM: Searching for the Optimal Primer Set Using a Novel Brute-Force Approach. Brute-force algorithms are typically slow, since they exhaustively search for the optimal solution among all possible solutions. Mutation Maker provides users with the option to precompute all potential primers in a specific temperature range and systematically search for the optimal primer set that fulfills the design criteria (Supplementary Figure S1).

First, the brute-force algorithm precomputes all possible forward and reverse primers around each mutation site with or without the use of Primer3.[33,34] The resultant forward and reverse primers are filtered based on their 3′ end length and are paired based on their overlapping region size. Next, the algorithm iteratively loops through the user-specified 3′ end and overlap T_m's, and for each T_m combination and for each mutation site it selects the best-scoring primer pair. The primer score is computed from deviations between true and expected values of the 3′ end and overlap T_m's, GC content, and the minimum 3′ end length. Depending on user input, the score function may also penalize primers for hairpins, homo-, and heterodimers. Lastly, for each set of primer pairs in each temperature step the algorithm computes a cumulative solution score and ultimately reports the best scoring primer set as its best solution.

SSSM: Novel Fast-Approximation Algorithm Generates Highly Efficient Mutagenic Primer Set. In contrast to the exhaustive approach that precomputes all potential primers for all mutation sites, the fast-approximation algorithm dynamically grows the primers in its search for a mutagenic primer set that least violates the design criteria (Supplementary Figure S2).

The approximation algorithm first iterates through a user-specified range of primers' overlap T_m's and for each mutation site it finds the shortest primers' overlap with a T_m that satisfies

https://dx.doi.org/10.1021/acssynbio.0c00542
ACS Synth. Biol. 2021, 10, 357−370

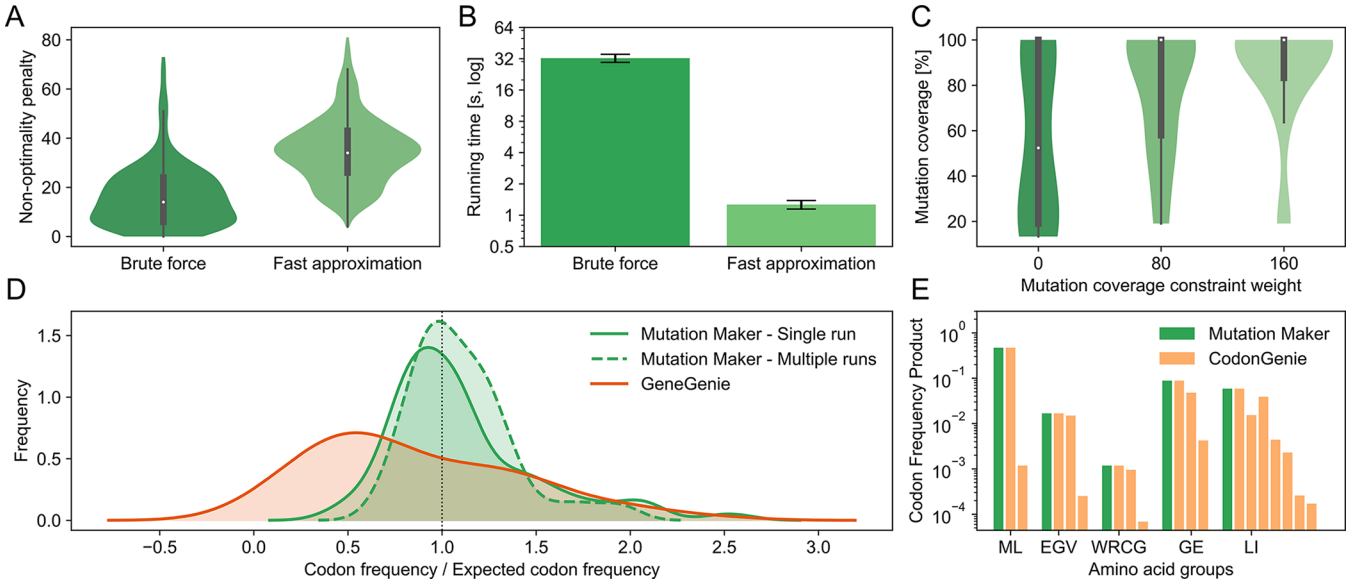


Figure 2. Distributions of (A) Site-scanning saturation mutagenesis (SSSM) cumulative design penalties ($\log_{10}$) for brute-force (dark green) and fast-approximation (light green) algorithms; (B) SSSM processing times ($\log_{10}$ seconds) for brute-force and fast-approximation designs (dark and light green, respectively). A total of 50 genes with 10 mutations were simulated. Wilcoxon Rank-sum was used to assess the difference between the two distributions ($p \sim 6.19 \times 10^{-92}$). (C) Distribution of mutation coverage for multi-site directed mutagenesis (MSDM) workflow as a function of user-adjustable weights. A total of 10 genes with 2 to 10 mutation sites with up to 5 mutations per site were simulated. *X*-axis (user-defined mutation coverage weights: 0, 80, and 160 dark to light green); *Y*-axis (percentage coverage of specified target sites). (D) Distribution of ratios of observed frequency over expected frequency of codons in protein sequences that were reverse translated and optimized by Mutation Maker CFP (codon frequency product) and GeneGenie CAI (codon adaptation index) codon optimization methods. A total of 5 random protein sequences (250 amino acids in length) were reversed translated by Mutation Maker, once (green, solid) or multiple times (green, dashed) and compared to GeneGenie's (orange). Mutation Maker's reverse translation incorporates randomization, therefore every protein sequence was reverse translated 5 times and results were summarized as shown. (E) Codon frequency product ($\log_{10}$ CFP) of degeneracy codons designed by Mutation Maker (green) and CodonGenie (orange) degeneracy algorithms. Random groups of amino acids were used as indicated. Note that CodonGenie outputs multiple solutions, whereas Mutation Maker only reports one degenerate solution with the highest CFP.

Table 1. Success Rates of Site-Scanning Saturation Mutagenesis (SSSM) Primer Sets Designed by Mutation Maker's Novel Fast-Approximation Approach for Gene X and Gene Y (~700 bp and ~1.2 kbp)[a]

library	number of target sites	clones tested	vector	host cell	success rate (%)
Gene X − 1	96	48	pAVE011-Kan	W3110	77.1
Gene X − 2	95	24	pAVE011-Kan	W3110	75
Gene X − 3	31	24	pAVE011-Kan	W3110	91.2
Gene X − 4	96	24	pET30a(+)	BL21(DE3)	78.3
Gene Y − 5	96	16	pET30a(+)	BL21(DE3)	81.3
Gene Y − 6	96	16	pET30a(+)	BL21(DE3)	81.3
Gene Y − 7	96	24	pET30a(+)	BL21(DE3)	75
				Average	**79.9**

[a]Libraries were cloned into different *E. coli* strains using different vectors as indicated. Clones were randomly picked and validated using Sanger Sequencing.

an arbitrary threshold. Then, it loops through a temperature grid that is derived from the user's input and it iteratively grows the forward and reverse primers (one base at a time) from the shortest overlap outward until the desired 3′ end T_m's are reached. Similar to the brute-force approach, this algorithm scores and ranks the primers, generating multiple candidate solutions, each containing a complete set of primer pairs for all mutation sites. The algorithm concludes by computing a cumulative score for each solution that ultimately determines the identity of the best solution.

An *in silico* comparison between the two SSSM algorithms revealed that primer sets produced by the fast-approximation method deviates significantly more from the optimal design criteria compared to the brute-force approach ($p < 0.05$; Figure 2A). Despite the computationally determined differences, laboratory results clearly indicate that the fast-approximation method generates functional primer sets with high success rates (on average 79.9%) across multiple genes, vectors and host cells (Table 1). Success rates fell within the expected efficiency range of amplification with Q5 Hot Start High-Fidelity DNA polymerase and gene assembly kit according to the product manual (New England Biolabs). Typically, pairs of mutagenic primers exhibited uniform amplification on a PCR plate following SOEing, producing a library with fairly even frequencies of target mutations. Given its significantly shorter computation time (Figure 2B; $p < 0.05$) and its high success rate in laboratory testing, we set the fast-approximation method as the default algorithm for SSSM.

MSDM: Combining Specific Amino Acid Changes Using Fast-Approximation Algorithm. Once beneficial

https://dx.doi.org/10.1021/acssynbio.0c00542
ACS Synth. Biol. 2021, 10, 357−370

Table 2. Success Rate of Multi Site-Directed Mutagenesis (MSDM) Primer Sets Designed by Mutation Maker's Novel Fast-Approximation Approach for Gene X (~700 bp)[a]

library	number of target mutations	vector	host cell	success rate (%)	average number of mutations per clone
Gene X − 1 L*	15	pET30a(+)	BL21(DE3)	68.8	2.6
Gene X − 1 M*	15	pET30a(+)	BL21(DE3)	75.0	2.9
Gene X − 1 H*	15	pET30a(+)	BL21(DE3)	81.3	3.6
Gene X − 2 L*	18	pET30a(+)	BL21(DE3)	61.5	2.5
Gene X − 2 M*	18	pET30a(+)	BL21(DE3)	81.3	2.0
Gene X − 2 H*	18	pET30a(+)	BL21(DE3)	64.3	2.8
Gene X − 3 L	14	pET30a(+)	BL21(DE3)	46.2	3.0
Gene X − 3 M	14	pET30a(+)	BL21(DE3)	71.4	1.2
Gene X − 3 H	14	pET30a(+)	BL21(DE3)	43.8	2.1
Gene X − 4 M	14	pET30a(+)	BL21(DE3)	87.5	2.9
Gene X − 5 L	13	pET30a(+)	BL21(DE3)	100	2.1
Gene X − 5 M	13	pET30a(+)	BL21(DE3)	81.3	2.5
Gene X − 5 H	13	pET30a(+)	BL21(DE3)	73.7	2.6
			Average	**72**	**2.5**

[a]Libraries were cloned into *E. coli* strain using vector as indicated. Clones were randomly picked and validated using Sanger sequencing. Low (L), Medium (M), and High (H) refers to the respective primer to template ratios interrogated (see Methods). *Processed by an earlier version of Mutation Maker (development version); few primers were manually adjusted to account for additional mutations.

mutations are identified by SSSM, scientists often seek to combine these single mutations in order to identify synergistic interactions that may enhance protein performance even further. Widely used commercial kits such as QCLM[17] offer simple and rapid protocols to perform MSDM. Briefly, QCLM generates combinatorial gene library *via* a multistep protocol that involves template denaturation, mutagenic primer annealing and extension, enzymatic DNA repair and template elimination and transformation (Figure 1B).

The minimal input for the MSDM algorithm is the GOI and a list of target residues alongside their respective amino acid changes. Users may specify the 5′ and 3′ flanking sequences to account for mutations at extreme gene ends. Similar to SSSM, the following primer parameters can be adjusted by the user: primer length and T_m intervals, 5′ and 3′ end length intervals, GC content, hairpins, and primer-dimers check and T_m consistency threshold across all primers. Users can also select the nonoverlapping option to design primers that can be annealed simultaneously. Without such an option, and under default settings, the algorithm may output partially overlapping primers for neighboring mutation sites that can be used only in successive rounds of MSDM experiments. To account for codon usage and to exclude rare codons users can specify the host organisms and a codon frequency threshold, respectively. Among the above constraints, only the lengths of the primer and its 5′ and 3′ ends serve as hard constraints that are never violated, while the remaining constraints are considered soft. Similar to SSSM, users can set the score function weights to control constraints violation and primer ranking. The MSDM workflow designs degenerate primers by default using a novel optimization algorithm described below. Once designed, unidirectional primers are displayed alongside their properties, including their computed ratios in the final reaction mix.

The MSDM algorithm first identifies all possible partitions of mutation sites, where each partition consists of groups of neighboring target sites that can be covered by a single primer, given the input constraints (Supplementary Figure S3). Then, for each mutation group within each partition and at each primer's T_m iteration, the algorithm combines selected codons that encode for the desired amino acids and computes all possible minimal primers around these mutation sites. Next, the 3′ ends of these minimal primers are then iteratively extended until they reach the desired T_m at each T_m step. Once grown, the algorithm computes scores for all primers in each mutation group and records the best scoring primer set at each iterated T_m. Upon completion of all iterations, the algorithm computes a cumulative solution score for all recorded solutions and ultimately outputs the best scoring primer set as its optimal solution. The MSDM algorithm may report an incomplete primer set that covers only some of the specified mutation sites, hence its cumulative score also accounts for mutation coverage. User-adjustable weights can be used to increase mutation coverage (Figure 2C). Experimental validation of MSDM primer sets revealed high success rates (72%) with an average of 2.5 mutations per clone (Table 2). According to the QCLM kit manual, success rates were in line with the expected mutagenesis efficiency for 3 simultaneous mutations (>55%). To conclude, Mutation Maker's MSDM workflow outputs quality primer sets that can produce diverse combinatorial gene libraries using the QCLM protocol.

***De Novo* Gene Synthesis: A Bundle of Novel Algorithms Enables Mutagenic Oligo Design.** PCR-based gene synthesis protocols such as PAS rely on short synthetic oligonucleotides that overlap one another by approximately 21 bases and have similar length (~60 bases) and overlap T_m (~56 °C).[19] The overlapping oligos cover the entire gene sequence and can be assembled into a full-length gene by high-fidelity PCR reactions (Figure 1C).

Although the principle underpinning gene synthesis is straightforward, computational design of overlapping oligos in practice can be challenging, especially the design of mutagenic oligos for gene diversification purposes. To ensure success of mutagenic gene synthesis experiments, oligos must exhibit unified length and T_m and mutations should not reside within their overlapping segments. In addition, other factors such as the sequence type (protein or DNA), mutation ratios, degeneracy, codon usage, and exclusion of hairpins, homodimers and certain motifs also contribute to the complexity of the design task. To address this challenge, we developed a bundle of novel interconnected optimization, constraint-satisfaction and backtracking algorithms that together find mutagenic oligos with unified length and T_m that also satisfy a

https://dx.doi.org/10.1021/acssynbio.0c00542
ACS Synth. Biol. 2021, 10, 357−370

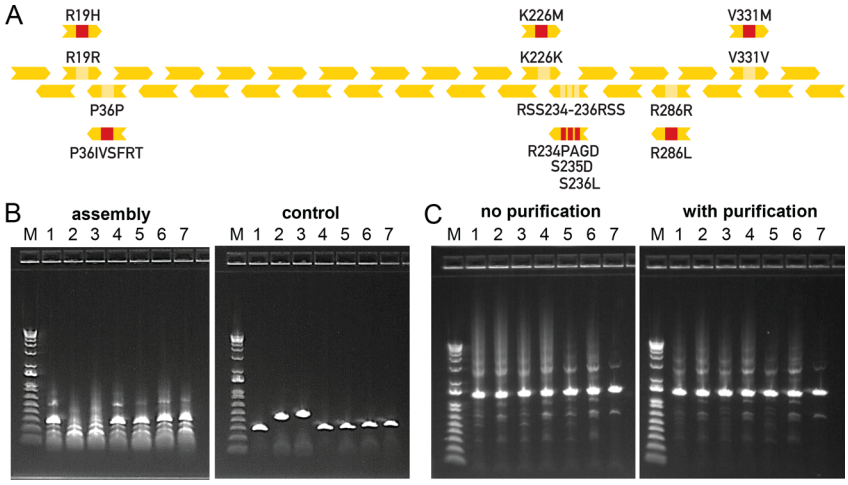


Figure 3. Experimental validation of mutagenic oligos for gene synthesis by PCR-based accurate synthesis (PAS) workflow. Degenerate oligos designed by Mutation Maker were successfully used to construct a full-length gene of ∼1.2 kbp in length. (A) Schematic representation of 32 overlapping fragments (yellow arrows 5′ and 3′ directions) designed by Mutation Maker, of which 6 fragments carried 16 mutations (red squares) at 8 different sites (as indicated). A total of 50 oligos were used to construct the gene library. (B, left to right) The first PCR to create gene segments. Assembly gel: Lane M, 1kb plus DNA ladder (ThermoFisher Scientific); lane 1, segment A1 (8 fragments, 309 bp); lane 2, segment A2 (12 fragments, 466 bp); lane 3, segment A3 (12 fragments, 497 bp); lane 4, segment B1 (8 fragments, 309 bp); lane 5, segment B2 (8 fragments, 315 bp); lane 6, segment B3 (8 fragments, 328 bp); lane 7, segment B4 (8 fragments, 338 bp). Control gel: controls corresponding to the segments form the assembly gel. (C, left to right) The 2nd PCR to create full-length gene. Gel without purification of segments from the 1st PCR (no purification): Lane M, 1 kb plus DNA ladder; lane 1, segments A1 + A2 + A3 (3 PAS); lane 2, segments B1 + B2 + B3 + B4 (4 PAS); lane 3, segments A1 + A2c + A3 (2 PAS + 1 control); lane 4, segments B1 + B2c + B3 + B4 (3 PAS + 1 control); lane 5, segments A1c + A2c + A3c (3 controls); lane 6, segments B1c + B2c + B3c + B4c (4 controls); lane 7, full-length control. Gel with agarose gel purified segments from 1st PCR (with purification) representing the same combination of segments as in the gel with no purification.

multitude of user-defined criteria. As elaborated below, the *de novo* gene synthesis workflow incorporates (i) a reverse translation algorithm that optimizes codon frequencies with respect to natural frequencies of the host organism; (ii) a constraint-satisfaction algorithm with a backtracking component that splits a given gene sequence into an even number of overlapping DNA fragments that conform to length, T_m, mutation location, GC content, hairpins, homodimers and motif exclusion constraints; (iii) a recursive degeneracy algorithm that outputs degenerate oligos.

***De Novo* Gene Synthesis: Solving Reverse Translation through Rigorous Codon Optimization.** To initiate overlapping oligo design for gene synthesis, users may input the GOI in a DNA or a protein sequence format. In case of the latter, reverse translation must be performed with respect to the native codon frequencies of the host organism to ensure efficient translation. Unfortunately, the level of inherent degeneracy in the genetic code makes reverse translation a nontrivial optimization problem, where a given protein sequence could give rise to numerous DNA sequences. To solve this problem, we developed a simplified optimization algorithm that softly optimizes for higher codon frequencies and considers GC content and motif exclusion. Briefly, Mutation Maker's reverse translation algorithm generates a DNA sequence from a set of randomly selected codons of the host organism (excluding rare codons). Thereafter, for that sequence the algorithm computes the codon frequencies product (CFP) which is proportional to codon adaptation index (CAI, see Methods).[35] Then the algorithm verifies that the sequence meets the design criteria in terms of GC content and motifs exclusion. If so, the algorithm appends the current solution to the list of possible solutions and records the highest CFP. These three basic steps are repeated until CFP cannot be significantly improved. Eventually, the solution with the highest CFP is reported as the optimal reverse translated sequence.

A comparison between Mutation Maker's CFP optimization algorithm and GeneGenie's CAI optimization[23] revealed a tighter distribution of codon frequencies around the natural frequencies of the host (Figure 2D). We therefore conclude that Mutation Maker's CFP optimization algorithm better reflects the natural frequencies in a given host organism.

***De Novo* Gene Synthesis: Constraint Satisfaction and Backtracking Algorithms Produce Functional Mutagenic Oligos.** The ultimate goal of the *de novo* gene synthesis algorithm is to split a given gene sequence into an even number of overlapping fragments that have consistent length and overlap T_m properties (Supplementary Figure S4). To do so, the algorithm iterates through a user-specified overlap T_m range and first groups the mutations that must reside on the same fragment. Then, around these mutation groups the algorithm generates minimal upstream and downstream sequences that mark the boundaries for gene fragmentation and can potentially serve as overlaps between consecutive gene fragments. The algorithm does so by growing these extensions until they satisfy the current overlap T_m and size constraints. At the next step, (while considering these fragmentation boundaries) a backtracking algorithm optimizes the split of the gene by optimizing the positions and the lengths of gene fragments until the entire gene sequence is covered. Briefly, starting from the 5′ end of the gene the backtracking algorithm adds one gene fragment at the time, placing it at the first possible position given the problem constraints. When the next added fragment cannot be placed due to constraint violation the algorithm shifts the last placed fragment. If the last fragment cannot be shifted anymore (due to constraints violation) the algorithm discards the last fragment and tries to shift the previous fragment and so on. Whenever the algorithm

https://dx.doi.org/10.1021/acssynbio.0c00542
ACS Synth. Biol. 2021, 10, 357−370

reaches a feasible solution that covers the entire gene sequence, it computes a complete solution score and records the highest. At each T_m iteration, the algorithm continues to search for the optimal gene split until it is found or until a timeout limit is reached. Once the algorithm completes iterating through the entire overlap T_m range, the best scoring set of fragments is selected. Next, the algorithm generates mutagenic oligos from these fragments by introducing and combining mutated or degenerate codons that respects the motifs exclusion constraint. At the very end, the algorithm computes the oligo ratios in the reaction mixture that are necessary to achieve the user-defined mutation frequency in the final library.

Degenerate oligos designed by Mutation Maker were successfully used to construct a full-length gene of 1.2 kb in length. A total of 32 overlapping fragments were designed and tested, of which 6 carried 16 mutations at 8 different sites (Figure 3A). The 32 overlapping oligos were split to 3 parts (segments A1, A2, and A3 consisting of 8, 12, and 12 fragments, respectively), or 4 parts (segments B1, B2, B3, and B4 consisting of 8 fragments each) for the first assembly PCR. The first PCR produced the expected gene segments for 8 fragments assembly (A1, and B1−B4) (300−340 nucleotides in length), but not for 12 fragments assembly (A2 and A3, >460 bases, Figure 3B). The second PCR produced full-length gene for both splits (A1−A3 and B1−B4), with or without gel purification and irrespective of the split strategy employed (Figure 3C). Together, indicating that small amount of successful first PCR product was sufficient to serve as a template for the second SOEing PCR, even though it was not detectable on agarose gel. Sequencing analysis revealed that all of the clones (32/32) corresponded to the full-length gene.

Taken together, the gene synthesis workflow features a bundle of novel algorithms for the design of mutated gene fragments that can be assembled into full-length gene variants and generate combinatorial gene libraries.

Novel Degeneracy Algorithms Solve Different Set Cover Problems. Given the scale of mutagenesis and gene synthesis experiments, scientists often seek to reduce the number of reactions in order to simplify the experimental procedure and lower its overall cost. One way of achieving that is to design degenerate primers that can introduce multiple mutations at a given site *via* a single reaction. Degenerate primers are essentially a mix of similar oligo sequences that only differ in specific positions and consequently give rise to a population of primers that cover a range of codon combinations.[36] Finding an optimal set of degenerate primers corresponds to a set cover problem, where the task is to cover a finite number of elements (amino acids) with as few as possible sets (represented by degenerate codons). Here, we developed two different degeneracy algorithms for two similar yet distinct set cover problems.

In the SSSM workflow, users can directly enter the desired degenerate codons or alternatively can select a set of amino acids that must be included in and/or must be excluded from the degenerate solution. Importantly, if users do choose to compute degenerate codons from inclusion and/or exclusion lists, amino acids that are not explicitly specified may also be included in the degeneracy solution, while only stop codons are excluded by default. To solve this first set cover problem, we developed a heuristic algorithm that first transforms the list of desired amino acids into a set of permuted codons, with one codon per amino acid (maximum number of permutations is predefined). Thereafter, the algorithm iterates over the list of permuted codons, while attempting to identify degenerate alphabets that encode the amino acids specified in the inclusion but not in the exclusion list. If no solution is found, the algorithm splits the list of codons into two subsets and repeatedly searches for a minimal set of degenerate codons in each subset. It does so recursively until the solution satisfies the user input. More specifically, for every iteration, the algorithm updates the current best solution, if the number of codons that are encoded by the degenerate alphabet is lower, and if the degenerate alphabet itself is shorter. The algorithm may report the results before it loops through all possible permutations, when the predefined time limit is reached or when it finds the optimal solution (*e.g.*, finds one degenerate codon that encodes exactly one codon per one amino acid from list of desired amino acids).

In MSDM and gene synthesis workflows the designed degeneracy solution is strictly restricted to the list of specified substitutions in the input. In addition, the solution must encode the parental amino acids at each site and may exclude rarely used codons. To solve this second set cover problem, we developed another heuristic algorithm that randomly selects codons for a given set of amino acids, computes a single degenerate codon for this set and decodes it back to the list of amino acids. This step is repeated multiple times and results are recorded. Then the algorithm retains only degenerate solutions that produced the lowest number of amino acids. From the resultant list, it then selects the solution with the highest CFP. The algorithm then checks whether the generated amino acids list matches the user input. If so, solution is reported. If not, the algorithm splits the set of amino acids into all possible subsets, and recursively repeats the previous steps for each subset, until the degeneracy codon fulfills the design criteria.

Importantly, heuristic algorithms are optimized for computational performance and by definition find a solution that might not be the optimal one. Hence, we next evaluated the performance of our degeneracy algorithm in terms of producing minimal sets of degenerate codons with high CFP. In this respect, Mutation Maker's degeneracy results were consistent with the highest scoring CodonGenie[37] degeneracy output (Figure 2E). Together we conclude that Mutation Maker enables design of degenerate primers that simplify the experimental procedure and consequently save time and cost.

■ DISCUSSION

Protein engineering will continue to transform the research landscape of biotherapeutics and industrial chemical processes. The ability to generate novel biotherapeutics with designer properties and to develop novel enzymatic processes[38,39] can be primarily attributed to decades of advancements in protein design and directed evolution techniques that provide new means to optimize protein performance or to create entirely new proteins.[1,40−42] Protein engineering techniques typically begin with a diverse library of gene variants in a test tube, followed by expression in a host organism and screening or selection for variants with desired properties. Gene diversification is therefore a key step that determines the quality of the library and consequently the success of identifying variants with beneficial mutations at a reasonable effort, time and cost. However, the scale and the complexity of gene diversification in practice demands the development of computational tools for automated library design that can significantly reduce experimental time and mitigate biases and errors. To address

https://dx.doi.org/10.1021/acssynbio.0c00542
ACS Synth. Biol. 2021, 10, 357−370

this issue, we developed Mutation Maker, a mutagenic oligo design software for SSSM, MSDM, and *de novo* gene synthesis workflows that are among the most frequently used gene diversification methodologies. Such protein engineering workflows are well established in the industry and are considered the most efficient procedures for rapid data generation.

The motivation to create Mutation Maker stemmed from the lack of any protein engineering-oriented application that enables bulk design of mutagenic oligos at industrial scale and supports multiple gene diversification modalities for generating random, directed and combinatorial gene libraries. Although several tools for mutagenic oligo design exist,[16,20−22] they are largely limited, obsolete or not freely available for widespread commercial use. For example, AMUSER[21] and HTP-oligo designer[20] were intended for site-directed mutagenesis, yet these tools permit only a single mutation per sequence, do not account for degeneracy and offer limited functionalities. Deepscan,[16] stands out from the freely available automated design tools, enabling mutagenic primer design at scale. Deepscan certainly represents a significant step forward for designing random gene diversification experiments. Nonetheless, it exhaustively generates primers for every codon in a given sequence and it still lacks many of the functionalities that Mutation Maker has to offer.

With respect to design of gene synthesis experiments, several tools[26] including DNAWorks[24,25] and GeneGenie[23] can effectively split genes into synthetic oligos. However, unlike Mutation Maker these tools were not designed to support large-scale protein engineering experiments. More specifically, DNAWorks is limited in terms of introducing multiple mutations, it requires multiple runs and many manual steps, deeming it unsuitable for high-throughput gene library design tasks. Similarly, GeneGenie[23] does not permit introduction of specific mutations, it only accepts protein sequences and it cannot handle large input sequences. Mutation Maker is therefore, to our best knowledge, the only software that enables automated design of combinatorial gene libraries by gene synthesis methodologies.

To ensure efficient translation of synthetic genes in the host organism Mutation Maker also features reverse translation functionality that is tightly coupled to codon optimization. In this regard several proprietary and freely available codon optimization algorithms have been developed.[23,26,43−46] However, many of the published methods optimize the codons in the target gene with respect to the CAI.[35] The CAI reflects the deviation of codon frequencies in a given sequence from those observed in a reference set of highly expressed genes in the target host organism. CAI highly correlates with gene expression, hence it has become a preferred optimization method.[26] Nevertheless, concern has been voiced that CAI maximization may lead to tRNA depletion and consequently to an increase in translational errors.[44] To mitigate such possibility, Mutation Makers offers an alternative CFP optimization method that is proportional to CAI, but only softly optimizes for higher codon frequency and therefore better reflects the native codons landscape in a given host. Translation efficiency can also be hampered by secondary loop structures along the mRNA sequence. In this respect, the reverse translation algorithm picks codons at random, which in turn significantly decreases the probability of generating long repetitive sequences that could lead to the formation of RNA-secondary loop structures.

Despite its many unique features Mutation Maker inevitably shares some functionalities with existing tools. However, while certain gene diversification features may be distributed across multiple tools and Web sites, Mutation Maker provides a "one-stop shop" platform for high-throughput protein engineering experiments. Furthermore, Mutation Maker's backbone is scalable, container-based, and open source, providing a robust foundation for heavy workload, future development, and further scope expansion by the greater scientific community.

The inherent complexity and plasticity of the novel algorithms developed here necessitated the development of a simplified user interface that maximizes users' ability to fine-tune key parameters and constraints in order to adapt to different experimental scenarios and protocols. Streamlining and simplifying the entire library design process was of paramount importance; therefore, we conducted an extensive user experience research with protein engineering scientists in order to guide Mutation Maker's architecture and design. To this end, Mutation Maker offers an up to date, dynamic and intuitive user interface that features tooltips, accepts multiple input formats, and includes interactive visualization, printing, export and results sharing functionalities.

The novel algorithms described here were optimized and validated by rigorous *in silico* simulations, which in turn helped to define the default settings of this tool. In addition, mutagenic oligos for all design workflows were experimentally validated in real-world industrial protein engineering projects. Nonetheless, it is important to note the limitations of these algorithms. For performance optimization reasons, the SSSM brute-force approach is only exhaustive within fairly large T_m windows that slide along the user-specified T_m range. Thus, in essence, not all possible primers across the specified T_m range are interrogated. Mutation Maker by design does not optimize for the presence of GC clamps throughout, since this constraint often eliminated better scoring solutions. The scoring functions for evaluating primers and solutions have been optimized based on past experience and initial laboratory testing but have not been optimized using exhaustive laboratory testing and therefore are subject to improvement over time. Mutation Maker's gene split algorithm can be also further optimized, since only simplified scores were used to evaluate fragments during backtracking as a trade-off for reduced computational time. The gene synthesis algorithm may not guarantee an optimal solution for complex cases when it hits the hardcoded timeout for each T_m iteration. It is worth noting that the algorithms for reverse translation and for degeneracy are solving these problems using heuristics and randomization and therefore may not output the optimal solution. Furthermore, the gene synthesis algorithm only checks for secondary structures at the fragment level and does not scan the entire reverse-translated sequence for resulting mRNA secondary structures that may affect translation efficiency. The incorporation of an additional codon juggling step following reverse translation as performed by GeneDesign[27] could further reduce the possibility of RNA-secondary loop structures formation.[47]

Despite the above limitations, laboratory test results were highly successful. Mutation Maker mutagenic oligos were successfully used to generate diverse random and combinatorial gene libraries. Finally, Mutation Maker is a freely available, open source software that can be deployed as web application on a server or locally. Mutation Maker was designed to be modular and extensible, aiming to encourage scientists to add

https://dx.doi.org/10.1021/acssynbio.0c00542
ACS Synth. Biol. 2021, 10, 357−370

new methods and protocols that can accelerate protein engineering and synthetic biology research and applications.

METHODS

Mutation Maker Features Scalable, Open Source, and Container-Based Architecture. Mutation Maker is composed of five distinct components as follows: (i) A JavaScript (JS) User Interface (UI) where users can submit oligo design tasks and visualize the results using a customized version of the neXtProt viewer,[48,49] (ii) A Worker that is assigned by the open source Celery asynchronous task queue, where computation takes place, (iii) An Application Programming Interfaces (API) that provide communication between the UI and the Worker, (iv) An open source Redis in-memory database that is used to temporarily store information about the design task and its respective results, (v) The AWS (Amazon Web Services) Lambda service is also optionally employed to parallelize part of the SSSM computation that utilizes Primer3 binary.[33,34] However, Mutation Maker can run on any computer that provides 256MB RAM for the UI, 512MB RAM for the API, and 8GB RAM for the Worker. All the aforementioned services can be executed in a local Python and JavaScript environment or alternatively *via* a Docker Compose script that executes each service as a Docker container. Note that parallelization of Primer3 under local deployment is possible without the need for an AWS account, yet oligo design run time with Primer3 is rather slow in this case.

User Interface. The web application is based on React, TypeScript and Ant Design JavaScript libraries. Landing page provides links to the three workflows (SSSM, MSDM and gene synthesis). Each workflow consists of an input form with advanced on-the-go validation and a dynamic result page. Results are displayed using interactive table, modified neXtProt Feature Viewer[48,49] and Excel export. Results with input parameters can be exported to PDF or as a temporary sharable link. Designed oligos can be exported as annotated XLSX file. The neXtProt Feature Viewer is JavaScript Library developed by Swiss Institute of Bioinformatics (SIB) to visualize protein sequence features using D3 design.[48,49] These libraries have been modified to meet the visualization requirements for our three design workflows. The following features were added: GC content graph, primer directionality, mutation alignment, and integration with interactive results table and with React user interface.

Melting Temperature Calculations, Primer Dimer and Hairpin Checks. Melting temperature was computed using Primer3 Python library[34] (available at https://pypi.org/project/primer3-py/), with the nearest neighbors Santa Lucia approach[50] and the following settings: calculation_method = "santalucia", salt_correction = "owczarzy", na = 50 (Na^+ ion concentrations, mM), k = 50 (K^+ ion concentration, mM), tris = 20 (Tris-HCL concentration, mM), mg = 2 (Mg^{2+} ion concentration, mM), dntp = 0.2 (dNTP concentration, mM), dnac1 = 500 (primer concentration, for PAS was set to simulated concentration of 583 nM), precision = 1 (precision of the temperature calculation in digits after the decimal point). For PAS oligo T_m's a T_m correction factor of 3 °C was applied. Concentrations of buffer components can be adjusted before installation as indicated in the readme file in the code repository (https://github.com/Merck/Mutation_Maker).

Primer hairpin, homo-, and heterodimer temperatures were computed using the same Primer3 Python library.[34] Hairpin and primer-dimer penalties were applied to candidate oligos only when primer-dimer or hairpin formation temperatures were very close to the specified optimal reaction temperature. The heterodimer penalty is considered only for SSSM and MSDM workflows, while for PAS heterodimer is omitted, given the desired overlap between consecutive oligos. In SSSM flanking primers were compared with each candidate mutagenic forward or reverse primer. In MSDM all possible heterodimers are considered, since all unidirectional primers are assumed to be used in a single reaction. For the SSSM brute-force approach option, primer-dimer and hairpin checks are performed by default using the Primer3 binary. Note that in order to enable longer primer design for SSSM using Primer3, the maximum primer length in Primer3 was increased from 36 to 60 nucleotides in the libprimer3.c library.

Score Functions. All workflows utilize score functions that are based on deviations of primers properties from the optimal values. For SSSM and MSDM the solution score consists of sum of scores for each individual primer and for gene synthesis the solution score consists of averaged score of all fragments and overlaps. The score for a single primer (or a fragment) is computed as a root sum of weighted squared errors for all parameters and can be generalized by this formula:

$$\text{Score} = \sqrt{\sum_{p \in \text{parameters}} w_p \cdot (\Delta v_p)^2}$$

where:

- p represents iterator over all parameters (melting temperature, primer length, GC content, *etc.*).
- w_p is a user specified weight for the parameter.
- Δv_p is the difference of actual parameter value from optimal value.

However, each workflow requires specific amendments to accurately evaluate candidate solution. Detailed description of scoring functions for each workflow are given below.

SSSM primer pair score:

$$\text{pair_score} = \sqrt{w_{\text{OT}_m} \cdot \Delta\text{OT}_m^2 + w_{T_m3} \cdot (\Delta T_m3_{fw}{}^2 + \Delta T_m3_{rw}{}^2) + w_{L3} \cdot (\Delta L3_{fw}{}^2 + \Delta L3_{rw}{}^2) + w_{GC} \cdot (\Delta GC_{fw}{}^2 + \Delta GC_{fw}{}^2)} + \sqrt{w_{HP} \cdot (\Delta HP_{fw}{}^2 + \Delta HP_{rw}{}^2) + w_{PD} \cdot (\Delta HD_{fw}{}^2 + \Delta HD_{rw}{}^2 + \Delta PD_{fw}{}^2 + \Delta PD_{rw}{}^2)}$$

where:

- ΔOT_m is the difference between the melting temperature of overlap and the PCR reaction temperature.
- ΔT_m3 is the difference between the temperature of the primer 3′ end and the specified minimal T_m of the 3′ end, for the forward (fw) and the reverse (rw) primers.
- $\Delta L3$ is the difference between the primer 3′ end size and the minimal 3′ end size.
- ΔGC is the deviation of the actual GC content from the specified GC content interval, in percentage.
- ΔHP is the difference between the hairpin melting temperature $T_{hairpin}$ (if the primer can create a hairpin), and the "safe self-binding temperature" T_{safe} which is

https://dx.doi.org/10.1021/acssynbio.0c00542
ACS Synth. Biol. 2021, 10, 357−370

defined as (PCR reaction temperature) − 2(T_m range size). If $T_{hairpin} < T_{safe}$, then this term is ignored.

- ΔHD is calculated the same as ΔHP, but for homodimers.
- ΔPD is calculated the same as ΔHP, but for dimers between the forward primer and the reverse flanking sequence (fw) and between the reverse primer and the forward flanking sequence (rw).
- w_X are user-supplied weights for each variable X (X = OT_m, T_m3, *etc.*).

SSSM solution score:

$$\sum_{i \in \text{pairs}} \text{pair_score}_i$$

MSDM primer score:

$$\text{primer_score} = \sqrt{\begin{array}{l} w_{T_m}{}^2 \cdot \Delta T_m{}^2 + w_{GC_O}{}^2 \cdot \Delta GC_O{}^2 + w_L{}^2 \cdot \Delta L^2 \\ \quad + w_{L3}{}^2 \cdot \Delta L3^2 \\ + w_{L5}{}^2 \cdot \Delta L5^2 + w_{HP}{}^2 \cdot \Delta HP^2 + w_{HD}{}^2 \cdot \Delta HD^2 \end{array}} + w_{HD} \cdot \sqrt{\sum_{i \in \text{heterodimers}} \Delta PD_i{}^2}$$

MSDM solution score:

$$\frac{\sum_{i \in \text{primers}} \text{primer_score}_i}{n_p} + w_{Cov} \cdot \text{Cov}$$

where parameters are as described for SSSM, with the following differences:

- ΔT_m is the difference between the primer melting temperature and the PCR reaction temperature.
- ΔGC_O is the difference between the actual and optimal GC content, in percentage.
- ΔL is the difference between the primer length and minimal primer length.
- ΔL5 is the difference between the primer 5′ end size and the minimal 5′ end size.
- ΔPD_i is the difference between the melting temperature of the i-th heterodimer (between the current primer and another primer), and the "safe" melting temperature, as in ΔHP used for SSSM.
- n_p is the number of primers.
- Cov is the coverage of mutations by the MSDM solution primers, in percentage.
- Other symbols have the same meaning as in the SSSM scores.

Gene synthesis fragment score:

$$\text{fragment_score} = \sqrt{w_{L_f} \cdot \Delta L_f{}^2 + w_{HP} \cdot \left(\sum_{i \in \text{hairpins}} \Delta HP_i^2 + \sum_{i \in \text{homodimers}} \Delta HD_i{}^2 \right)}$$

Gene synthesis overlap score:

$$\text{overlap_score} = \sqrt{w_{L_O} \cdot \Delta L_O{}^2 + w_{GC_O} \cdot \Delta GC_O{}^2 + w_{T_O} \cdot \Delta T_O{}^2}$$

Gene synthesis solution score:

$$\left(\sum_{i \in \text{overlaps}} \text{overlap_score}_i + \sum_{i \in \text{fragments}} \text{fragment_score}_i \right) / n_f$$

where:

- ΔL_f is the difference between the actual length of a fragment and the optimal length.
- ΔHP_i is the value of ΔHP (as described for the SSSM formulas) for the i-th hairpin region in a given PAS fragment. The PAS workflow considers multiple hairpins for the same DNA fragment, as PAS fragments are typically much longer that SSSM or MSDM primers.
- ΔHD_i is the value of ΔHD (as described for the SSSM formulas) for the i-th region in a given PAS fragment which can take a part in forming a homodimer.
- ΔL_O is a difference between the length of the overlap of two fragments and the optimal length of the overlap.
- ΔGC_O is the difference between GC content for the overlap and the specified optimal GC content.
- ΔT_O is the difference between overlap temperature and the optimal overlap temperature.
- n_f is the number of fragments.
- Other symbols have the same meaning as in the SSSM scores.

The solution score is the average of scores for all fragments and overlaps. The weights used in the score functions are hardcoded. The Mutation Maker uses the following values:

$w_{L_O} = w_{GC_O} = w_{T_O} = w_{L_f} = 1$.

$w_{HP} = 2$.

Backtracking Algorithm. The backtracking algorithm of the gene synthesis workflow can be described by the following pseudocode:

```
procedure bt(c)
  if reject(P,c) then return
    else record(c)
  if accept(P,c) then
    save_best_solution(c)
    return
  s ← first(P,c)
  while s ≠ NULL do
    bt(s)
    s ← next(P,s)
```

where:

Input: "**c**" is an ordered list of starts of fragments in a partial solution consisting of DNA fragments covering an initial part of the gene. **P**: is the input data for the gene synthesis algorithm (GOI, constraints *etc.*) plus selected T_{min} for the gene synthesis workflow. **reject**(P,c) is a function that returns true, if new partial solution "c" violates any of the following conditions:

(1) Constraints given by P.
(2) The last fragment from the sequence of "c" must not form a hairpin or homodimer for given T_m.
(3) The algorithm has not already found a partial solution that ends at the same nucleotide, has the same parity of

https://dx.doi.org/10.1021/acssynbio.0c00542
ACS Synth. Biol. 2021, 10, 357−370

the number of fragments (odd, even), and has a lower value of the score function than solution "c".

Condition 3 refers to a dynamic programming optimization, where the score function serves as the optimization criterion and the state is defined by the end position of fragment sequence and parity of the fragment sequence length. Function **record** (c) saves partial solution "c" (list of fragments) and its score in a table with index given by pair (last nucleotide position, parity). The **accept**(P,c) function checks that (i) the list of fragments "c" covers the whole gene, (ii) the number of fragments is even, (iii) the score of solution "c" is better than the best recorded score for complete solution (constraints "P" are used for computing the score in **accept**()), (iv) the last fragment from "c" does not form a hairpin or homodimer for given T_m. The **save_best_solution**(c) function records sequence "c" and its respective score. Function **first**(P,c) extends partial solution "c" by adding a DNA fragment with the length closest to the optimal fragment length, given constraints "P". Finally the **next**(P,s) function replaces the last fragment in "s" by the next best fragment with respect to the distance from the optimal fragment length that fulfills constraints "P".

Codon Usage and Product of Codon Frequencies Optimization. To account for species-specific codon usage in the MSDM and gene synthesis workflows Mutation Maker utilizes taxa-specific genetic code tables and species-specific raw codon frequencies. The 33 taxa-specific genetic code tables are available from https://www.ncbi.nlm.nih.gov/Taxonomy/Utils/wprintgc.cgi, while 35 799 species-specific raw sum of codons are available from ftp://ftp.ebi.ac.uk/pub/databases/cutg/.[51] Codon usage bias is defined as the difference in the frequency of synonymous codons in individual species. These relative codon frequencies can be directly computed from the raw sum of codons in a given species. Mutation Maker's codon usage function simply uses the standard species taxonomy ID to retrieve the corresponding genetic code table and the respective raw codon frequencies to calculate these relative frequencies.

Mutation Maker's codon optimization approach softly optimizes for the codon frequencies product (CFP) which is proportional to CAI:

$$\mathrm{CAI} = \left(\frac{\prod_{i=1}^{L} f_i}{\prod_{i=1}^{L} \max_j f_j^{(i)}}\right)^{1/L} = \left(\prod_{i=1}^{L} \max_j(f_j^{(i)})\right)^{-1/L} \cdot \left(\prod_{i=1}^{L} f_i\right)^{1/L} = C\cdot\left(\prod_{i=1}^{L} f_i\right)^{1/L} = C\cdot(\mathrm{CFP})^{1/L}$$

where $f_j^{(i)}$ are frequencies for synonymous codons for amino acid i, C is a constant for a given gene sequence, and $\mathrm{CFP} = \prod_{i=1}^{L} f_i$.

Better reflected when log is taken for both sides of equation:

$$\log \mathrm{CAI} = C' + \frac{1}{L}\cdot\sum_{i=1}^{L} \log f_i = C' + \frac{1}{L}\log \mathrm{CFP}$$

where $C' = \log C$.

Ratio Calculation for Gene Synthesis. For gene synthesis, users can specify the percentages of occurrence of each mutation in the resulting mixture. This is achieved by setting concentrations of all variants of the DNA fragments generated by the gene splitting algorithm. The desired concentrations of the variants (referred to as "oligos" here) are computed by the following algorithm:

Let $r_1^{(i)}, \ldots, r_k^{(i)}$ be the desired ratios for mutations $M_1, \ldots, M_k$ at the i-th mutation site on the fragment.

We can suppose that

$$\sum_j r_j^{(i)} = 1$$

while one can rephrase the problem as follows:

Find a concentration for each oligo, as a [0, 1] share of the total number of molecules, which gives a mixture with the following property: If a molecule is randomly picked from the mixture, the probability that it has mutation M_j at the i-th mutation site is equal to $r_j^{(i)}$. The computation of the oligo concentrations is done according to the following theoretical result:

Suppose that an oligo has mutations $M_{j1}^{(1)}, \ldots, M_{jn}^{(n)}$ at mutation sites 1, ..., n, where n is the number of mutation sites on the fragment. Then this oligo share must be equal to $r_{j1}^{(1)} \ldots r_{jn}^{(n)}$.

Proof. Let $A_1, \ldots, A_n$ be random variables, the values of which are mutations at sites 1, ..., n.

Let us assume that these variables are *independent*. Then,

$$P(A_i = M_p^{(i)} \wedge A_j = M_q^{(j)}) = P(A_i = M_p^{(i)}) \cdot P(A_j = M_q^{(j)}).$$

Assuming a random process in which the next oligo to be put in the mixture is repetitively selected from a set of all possible mutated oligos.

If an oligo with a specific mutation combination with probability equal to the product of probabilities for its mutations is selected, then, indeed, the same probability for randomly picking this oligo from the resulting mixture is obtained.

Note that the above statement does not prove that this is the only possibility to achieve the desired mutation ratios. However, it guarantees that this requirement will be satisfied, as long as all possible mutation combinations can be used. For degenerate solution, when an oligo specification can cover multiple mutations at the same site, the solution may not work. In this case, Mutation Maker outputs the concentrations of the affected oligos as an "unknown" value. However, if the ratio of the mutations generated by one degenerate codon is as required (for instance, 1:1), then the same method can be used also for oligos with degenerate codons. If this is the case, then the algorithm just uses a sum of expected ratios for mutations covered by the degenerate codon instead of values for individual mutations as shown in the product above. Another condition that must be met is that the degenerate codons should not overlap for the same mutation site (for instance, degenerate codons for FL and LI should not reside on different oligos for the same site). Otherwise, independence of the random variables as shown in the proof above cannot be assumed.

Experimental Validation for SSSM. The SSSM primer sets designed by the fast-approximation for gene X and gene Y (~700 bp and ~1.2 kbp in length) were validated as follows. The SSSM library was created by a modified overlap extension PCR method.[31,52] Typically, up to 96 amino acids in GOI sequence were selected as a set of target sites for one library, and forward ("NNK") and reverse ("MNN") mutagenic primers were designed using the SSSM workflow with default settings. The resultant forward and reverse primers were

https://dx.doi.org/10.1021/acssynbio.0c00542
ACS Synth. Biol. 2021, 10, 357−370

synthesized and formatted into two 96-well plates separately (Integrated DNA Technology, USA). The GOIs was cloned into pET30a(+) or pAVE011-Kan vector and used as the template for PCR. To create N- and C-terminus mutagenic fragments, the first PCR reactions were performed with Q5-Hot Start DNA polymerase (New England Biolabs, USA) using reverse and forward primer plates paired with the 5′- and 3′ flanking primers, respectively. The following PCR conditions were used: initial denaturation at 98 °C for 1 min; 25 cycles of [98 °C for 10 s; 60 °C for 20 s; 72 °C for 20−30 s/kb]; and final extension at 72 °C for 2 min. Then, the N- and C-terminus fragments were mixed together in a single 96-well PCR plate and treated with ExoSAP-IT (Thermo-Fisher, USA) and DpnI to digest the parental plasmid and primers. To create full-length mutant gene, the second PCR (SOEing PCR) reactions were performed using ExoSAP-IT/DpnI treated fragment mixture and 5′- and 3′- flanking primers under the same conditions used for the first PCR. After SOEing, amplifications in each well were confirmed by running on 96-well E-gel system (Life Technologies, USA). The mutant GOI fragments were pooled to give a single gene library and cloned back into the plasmid by NEBuilder HiFi assembly kit (New England Biolabs, USA) and transformed to *E. coli* cells. Clones were randomly picked, and the mutations were validated using Sanger Sequencing (GeneWiz, USA).

Experimental Validation for MSDM. MSDM primer sets designed by Mutation Maker were validated as follows. Gene X (∼700 bp in length) was cloned into pET30a vector. The library targeted combinations of approximately 13 to 18 mutations ranging between 6 and 12 sites per library. The MSDM library was created using the QuikChange Lightning multiple-site-directed mutagenesis kit (Agilent Technologies, USA). In short, site-directed primers were designed using MSDM workflow with default settings. The resultant primers were synthesized by Integrated DNA Technology (USA). The primers encoding mutations were mixed in a single tube and used for QuikChange reaction. To confirm the robustness of the protocol, the reaction was performed under three different conditions; Low (L), Medium (M), and High (H), in which 6, 15, and 30 nmol of mutagenic primer mix were used, respectively, against 30 ng of the template plasmid in 15 μL reaction volume. The resultant libraries transformed into *E. coli* BL21(DE3) cells, and isolated colonies were tested by Sanger sequencing (GeneWiz, USA) to confirm library quality.

Experimental Validation Gene Synthesis (PAS). The complete synthetic gene library of the test GOI (gene Y, ∼1.2 kbp) was constructed using the PAS protocol described by Xiong *et al.*[19] Briefly, Mutation Maker PAS workflow (under default settings) split the ∼1.2 kb gene into 32 overlapping fragments, of which 6 fragments carried 16 mutations at 8 target sites, producing 50 oligo primers in total (Figure 3C). The first PCR assembly of the 8 or 12 fragments and the second SOEing PCR assembly of 3 or 4 subfragments were performed using Q5-Hot Start DNA polymerase (New England Biolabs, USA). Following each PCR round, the expected bands were purified by agarose gel extraction using ADB buffer and DCC-5 column (Zymo Research, USA). The final library fragments were cloned into pET30a(+) vector using NEBuilder HiFi Gene Assembly kit and transformed *E. coli* BL21(DE3) cells, then the isolated colonies were tested by Sanger sequencing (GeneWiz, USA) to confirm library quality.

■ ASSOCIATED CONTENT

Supporting Information

The Supporting Information is available free of charge at https://pubs.acs.org/doi/10.1021/acssynbio.0c00542.

Figures S1−S4 illustrate Mutation Maker algorithms (PDF)

■ AUTHOR INFORMATION

Corresponding Author

Danny A. Bitton − *R&D Informatics Solutions, MSD Czech Republic s.r.o., 150 00 Prague, Czech Republic*; orcid.org/0000-0003-3282-1017; Email: danny.bitton@merck.com

Authors

Kaori Hiraga − *Protein Engineering, MRL, Merck & Co. Inc., Rahway, New Jersey 07065, United States*

Petr Mejzlik − *AI & Big Data Analytics, MSD Czech Republic s.r.o., 150 00 Prague, Czech Republic*

Matej Marcisin − *R&D Informatics Solutions, MSD Czech Republic s.r.o., 150 00 Prague, Czech Republic*

Nikita Vostrosablin − *AI & Big Data Analytics, MSD Czech Republic s.r.o., 150 00 Prague, Czech Republic*

Anna Gromek − *R&D Informatics Solutions, MSD Czech Republic s.r.o., 150 00 Prague, Czech Republic*

Jakub Arnold − *AI & Big Data Analytics, MSD Czech Republic s.r.o., 150 00 Prague, Czech Republic*

Sebastian Wiewiora − *AI & Big Data Analytics, MSD Czech Republic s.r.o., 150 00 Prague, Czech Republic*

Rastislav Svarba − *R&D Informatics Solutions, MSD Czech Republic s.r.o., 150 00 Prague, Czech Republic*

David Prihoda − *R&D Informatics Solutions, MSD Czech Republic s.r.o., 150 00 Prague, Czech Republic; Department of Informatics and Chemistry, Faculty of Chemical Technology, University of Chemistry and Technology, 166 28 Prague, Czech Republic*

Kamila Clarova − *R&D Informatics Solutions, MSD Czech Republic s.r.o., 150 00 Prague, Czech Republic; Department of Informatics and Chemistry, Faculty of Chemical Technology, University of Chemistry and Technology, 166 28 Prague, Czech Republic*

Ondrej Klempir − *R&D Informatics Solutions, MSD Czech Republic s.r.o., 150 00 Prague, Czech Republic*

Josef Navratil − *R&D Informatics Solutions, MSD Czech Republic s.r.o., 150 00 Prague, Czech Republic*

Ondrej Tupa − *R&D Informatics Solutions, MSD Czech Republic s.r.o., 150 00 Prague, Czech Republic*

Alejandro Vazquez-Otero − *R&D Informatics Solutions, MSD Czech Republic s.r.o., 150 00 Prague, Czech Republic*

Marcin W. Walas − *R&D Informatics Solutions, MSD Czech Republic s.r.o., 150 00 Prague, Czech Republic*

Lukas Holy − *R&D Informatics Solutions, MSD Czech Republic s.r.o., 150 00 Prague, Czech Republic*

Martin Spale − *R&D Informatics Solutions, MSD Czech Republic s.r.o., 150 00 Prague, Czech Republic*

Jakub Kotowski − *AI & Big Data Analytics, MSD Czech Republic s.r.o., 150 00 Prague, Czech Republic*

David Dzamba − *R&D Informatics Solutions, MSD Czech Republic s.r.o., 150 00 Prague, Czech Republic*

Gergely Temesi − *R&D Informatics Solutions, MSD Czech Republic s.r.o., 150 00 Prague, Czech Republic*

Jay H. Russell − *Protein Engineering, MRL, Merck & Co. Inc., Rahway, New Jersey 07065, United States*

Nicholas M. Marshall − *Protein Engineering, MRL, Merck & Co. Inc., Rahway, New Jersey 07065, United States*
Grant S. Murphy − *Protein Engineering, MRL, Merck & Co. Inc., Rahway, New Jersey 07065, United States*

Complete contact information is available at:
https://pubs.acs.org/10.1021/acssynbio.0c00542

Author Contributions

[⊥]K. Hiraga and P. Mejzlik contributed equally.

Author Contributions

DB and KH conceived the study. DB supervised the study. AG gathered the requirements from the scientists and coordinated the project. PM led the development and designed the MSDM and gene synthesis algorithms, and JA designed the SSSM workflows. PM, JA, NV, and MM implemented the algorithm and wrote the associated tests. AV wrote the gene synthesis simulation tests. NV designed and implemented the reverse translation and degeneracy algorithms for MSDM and gene synthesis and implemented oligo generation and ratio calculation for PAS. RS designed and implemented the degeneracy algorithms for SSSM. LH conducted the user experience research and designed the user interface. SW, MW, RS, MM, DP, and JA implemented the user interface. MM, JA, OK, MS, and DP packaged the software, and MS prepared it for open source. PM, MM, JN, and OT designed and implemented the score functions throughout. KC implemented the multispecies codon frequencies functions. JK, GT, DD, NM, and JR advised throughout the study. KH, DP, DD, MM, OK, and DB designed the figures. DP did the artwork. DB wrote the manuscript. PM, MM, NV, KH, KC, SW, and GM also contributed to the final draft. All authors read and approved the final version of the manuscript.

Notes

The authors declare no competing financial interest.
All code for this publication is available in the following GitHub repository: https://github.com/Merck/Mutation_Maker.

ACKNOWLEDGMENTS

We thank Keith A. Canada, Sandra K. Tremps, Jacob Lustig, Carol Rohl, Farida Kopti, Martin Ryzl, and Jens Christensen for their continuous support of this project. We thank Xinnian Li for his help in gathering the project requirements. We are also immensely grateful to Jiri Marsicin and Jan Tkacik who initiated development of the tool and to Jan Barta for his help implementing some of the user interface features. We also thank Christina Ferraro and Wai Ling Cheung-Lee for their help with the experimental validation. We would like to acknowledge and thank Jyoti Shah, Geoffrey Hannigan, and Victor Neduva for reviewing an earlier version of the manuscript. This work was supported by Merck Sharp & Dohme Corp., a subsidiary of Merck & Co., Inc., Kenilworth, NJ, USA.

ABBREVIATIONS

SSM, site saturation mutagenesis; SSSM, site-scanning saturation mutagenesis; MSDM, multi site-directed mutagenesis; PAS, PCR-based accurate synthesis; T_m, melting temperature; QCLM, QuikChange lightning multi site-directed mutagenesis; SOEing PCR, splicing by overlap extension polymerase chain reaction; GOI, gene of interest; CFP, codon frequencies product; CAI, codon adaptation index.

REFERENCES

(1) Hecht, M. H., Zarzhitsky, S., Karas, C., and Chari, S. (2018) Are Natural Proteins Special? Can We Do That? *Curr. Opin. Struct. Biol. 48*, 124−132.
(2) Regan, L., and DeGrado, W. F. (1988) Characterization of a Helical Protein Designed from First Principles. *Science 241*, 976−978.
(3) Hecht, M. H., Richardson, J. S., Richardson, D. C., and Ogden, R. C. (1990) De Novo Design, Expression, and Characterization of Felix: A Four-Helix Bundle Protein of Native-like Sequence. *Science 249*, 884−891.
(4) Kamtekar, S., Schiffer, J. M., Xiong, H., Babik, J. M., and Hecht, M. H. (1993) Protein Design by Binary Patterning of Polar and Nonpolar Amino Acids. *Science 262*, 1680−1685.
(5) Moore, J. C., and Arnold, F. H. (1996) Directed Evolution of a Para-Nitrobenzyl Esterase for Aqueous-Organic Solvents. *Nat. Biotechnol. 14*, 458−467.
(6) Dahiyat, B. I., and Mayo, S. L. (1997) De Novo Protein Design: Fully Automated Sequence Selection. *Science 278*, 82−87.
(7) Kuhlman, B., Dantas, G., Ireton, G. C., Varani, G., Stoddard, B. L., and Baker, D. (2003) Design of a Novel Globular Protein Fold with Atomic-Level Accuracy. *Science 302*, 1364−1368.
(8) Cravens, A., Payne, J., and Smolke, C. D. (2019) Synthetic Biology Strategies for Microbial Biosynthesis of Plant Natural Products. *Nat. Commun. 10*, 1−12.
(9) Neylon, C. (2004) Chemical and Biochemical Strategies for the Randomization of Protein Encoding DNA Sequences: Library Construction Methods for Directed Evolution. *Nucleic Acids Res. 32*, 1448−1459.
(10) Tee, K. L., and Wong, T. S. (2017) Back to Basics: Creating Genetic Diversity. In *Directed Enzyme Evolution: Advances and Applications* (Alcalde, M., Ed.), pp 201−227, Springer International Publishing.
(11) Mate, D. M., Gonzalez-Perez, D., Mateljak, I., Gomez de Santos, P., Vicente, A. I., and Alcalde, M. (2017) Chapter 8 - The Pocket Manual of Directed Evolution: Tips and Tricks. In *Biotechnology of Microbial Enzymes: Production, Biocatalysis and Industrial Applications* (Brahmachari, G., Ed.), pp 185−213, Academic Press.
(12) Qu, G., Li, A., Acevedo-Rocha, C. G., Sun, Z., and Reetz, M. T. (2020) The Crucial Role of Methodology Development in Directed Evolution of Selective Enzymes. *Angew. Chem., Int. Ed. 59*, 13204−13231.
(13) Miyazaki, K., and Arnold, F. H. (1999) Exploring Nonnatural Evolutionary Pathways by Saturation Mutagenesis: Rapid Improvement of Protein Function. *J. Mol. Evol. 49*, 716−720.
(14) Georgescu, R., Bandara, G., and Sun, L. (2003) Saturation Mutagenesis. In *Directed Evolution Library Creation: Methods in Molecular Biology* (Arnold, F. H., and Georgiou, G., Eds.), pp 75−83, Humana Press, Totowa, NJ.
(15) Siloto, R. M. P., and Weselake, R. J. (2012) Site Saturation Mutagenesis: Methods and Applications in Protein Engineering. *Biocatal. Agric. Biotechnol. 1*, 181−189.
(16) Acevedo-Rocha, C. G., Ferla, M., and Reetz, M. T. (2018) Directed Evolution of Proteins Based on Mutational Scanning. In *Protein Engineering: Methods in Molecular Biology* (Bornscheuer, U., and Höhne, M., Eds.), pp 87−128, Humana Press, New York, NY.
(17) Hogrefe, H. H., Cline, J., Youngblood, G. L., and Allen, R. M. (2002) Creating Randomized Amino Acid Libraries with the Quikchange® Multi Site-Directed Mutagenesis Kit. *BioTechniques 33*, 1158−1165.
(18) Xiong, A. S., Peng, R. H., Zhuang, J., Gao, F., Li, Y., Cheng, Z. M., and Yao, Q. H. (2008) Chemical Gene Synthesis: Strategies, Softwares, Error Corrections, and Applications. *FEMS Microbiol. Rev. 32*, 522−540.
(19) Xiong, A. S., Yao, Q. H., Peng, R. H., Duan, H., Li, X., Fan, H. Q., Cheng, Z. M., and Li, Y. (2006) PCR-Based Accurate Synthesis of Long DNA Sequences. *Nat. Protoc. 1*, 791−797.
(20) Camilo, C. M., Lima, G. M. A., Maluf, F. V., Guido, R. V. C., and Polikarpov, I. (2016) HTP-OligoDesigner: An Online Primer

Design Tool for High-Throughput Gene Cloning and Site-Directed Mutagenesis. *J. Comput. Biol. 23*, 27−29.
(21) Genee, H. J., Bonde, M. T., Bagger, F. O., Jespersen, J. B., Sommer, M. O. A., Wernersson, R., and Olsen, L. R. (2015) Software-Supported User Cloning Strategies for Site-Directed Mutagenesis and DNA Assembly. *ACS Synth. Biol. 4*, 342−349.
(22) Novoradovsky, A., Zhang, V., Ghosh, M., Hogrefe, H., Sorge, J., and Gaasterland, T. (2005) Computational Principles of Primer Design for Site Directed Mutagenesis. *NSTI Nanotech 2005, NSTI Nanotechnol. Conf. Trade Show*, 532−535.
(23) Swainston, N., Currin, A., Day, P. J., and Kell, D. B. (2014) GeneGenie: Optimized Oligomer Design for Directed Evolution. *Nucleic Acids Res. 42*, W395−400.
(24) Hoover, D. (2012) Using DNAWorks in Designing Oligonucleotides for PCR-Based Gene Synthesis. In *Gene Synthesis: Methods in Molecular Biology* (Peccoud, J., Ed.), pp 215−223, Humana Press.
(25) Hoover, D. M. (2002) DNAWorks: An Automated Method for Designing Oligonucleotides for PCR-Based Gene Synthesis. *Nucleic Acids Res. 30*, e43.
(26) Gould, N., Hendy, O., and Papamichail, D. (2014) Computational Tools and Algorithms for Designing Customized Synthetic Genes. *Front. Bioeng. Biotechnol. 2*, 41−41.
(27) Richardson, S. M., Wheelan, S. J., Yarrington, R. M., and Boeke, J. D. (2006) GeneDesign: Rapid, Automated Design of Multikilobase Synthetic Genes. *Genome Res. 16*, 550−556.
(28) Bode, M., Khor, S., Ye, H., Li, M.-H., and Ying, J. Y. (2009) TmPrime: Fast, Flexible Oligonucleotide Design Software for Gene Synthesis. *Nucleic Acids Res. 37*, W214−W221.
(29) Rouillard, J.-M., Lee, W., Truan, G., Gao, X., Zhou, X., and Gulari, E. (2004) Gene2Oligo: Oligonucleotide Design for in Vitro Gene Synthesis. *Nucleic Acids Res. 32*, W176−W180.
(30) Rydzanicz, R., Zhao, X. S., and Johnson, P. E. (2005) Assembly PCR Oligo Maker: A Tool for Designing Oligodeoxynucleotides for Constructing Long DNA Molecules for RNA Production. *Nucleic Acids Res. 33*, W521−W525.
(31) Horton, R. M., Hunt, H. D., Ho, S. N., Pullen, J. K., and Pease, L. R. (1989) Engineering Hybrid Genes without the Use of Restriction Enzymes: Gene Splicing by Overlap Extension. *Gene 77*, 61−68.
(32) Sayous, V., Lubrano, P., Li, Y., and Acevedo-Rocha, C. G. (2020) Unbiased Libraries in Protein Directed Evolution. *Biochim. Biophys. Acta, Proteins Proteomics 1868*, 140321.
(33) Rozen, S., and Skaletsky, H. (2000) Primer3 on the WWW for General Users and for Biologist Programmers. In *Bioinformatics Methods and Protocols: Methods in Molecular Biology* (Misener, S., and Krawetz, S. A., Eds.), pp 365−386, Humana Press, Totowa, NJ.
(34) Untergasser, A., Cutcutache, I., Koressaar, T., Ye, J., Faircloth, B. C., Remm, M., and Rozen, S. G. (2012) Primer3—New Capabilities and Interfaces. *Nucleic Acids Res. 40*, e115.
(35) Sharp, P. M., and Li, W. H. (1987) The Codon Adaptation Index-a Measure of Directional Synonymous Codon Usage Bias, and Its Potential Applications. *Nucleic Acids Res. 15*, 1281−1295.
(36) Iserte, J. A., Stephan, B. I., Goñi, S. E., Borio, C. S., Ghiringhelli, P. D., and Lozano, M. E. (2013) Family-Specific Degenerate Primer Design: A Tool to Design Consensus Degenerated Oligonucleotides. *Biotechnol. Res. Int. 2013*, 383646.
(37) Swainston, N., Currin, A., Green, L., Breitling, R., Day, P. J., and Kell, D. B. (2017) CodonGenie: Optimised Ambiguous Codon Design Tools. *Peer J. Comput. Sci. 2017*, e120.
(38) Sheldon, R. A., and Woodley, J. M. (2018) Role of Biocatalysis in Sustainable Chemistry. *Chem. Rev. 118*, 801−838.
(39) Schmid, A., Dordick, J. S., Hauer, B., Kiener, A., Wubbolts, M., and Witholt, B. (2001) Industrial Biocatalysis Today and Tomorrow. *Nature 409*, 258−268.
(40) Arnold, F. H. (2018) Directed Evolution: Bringing New Chemistry to Life. *Angew. Chem., Int. Ed. 57*, 4143−4148.
(41) Arnold, F. H. (2015) The Nature of Chemical Innovation: New Enzymes by Evolution. *Q. Rev. Biophys. 48*, 404−410.
(42) Turner, N. J. (2009) Directed Evolution Drives the next Generation of Biocatalysts. *Nat. Chem. Biol. 5*, 567−573.
(43) Raab, D., Graf, M., Notka, F., Schödl, T., and Wagner, R. (2010) The GeneOptimizer Algorithm: Using a Sliding Window Approach to Cope with the Vast Sequence Space in Multiparameter DNA Sequence Optimization. *Syst. Synth. Biol. 4*, 215−225.
(44) Chung, B. K. S., and Lee, D. Y. (2012) Computational Codon Optimization of Synthetic Gene for Protein Expression. *BMC Syst. Biol. 6*, 134.
(45) Tuan-Anh, T., Ly, L. T., Viet, N. Q., and Bao, P. T. (2017) Novel Methods to Optimize Gene and Statistic Test for Evaluation - an Application for Escherichia Coli. *BMC Bioinf. 18*, 100.
(46) Liu, X., Deng, R., Wang, J., and Wang, X. (2014) COStar: A D-Star Lite-Based Dynamic Search Algorithm for Codon Optimization. *J. Theor. Biol. 344*, 19−30.
(47) Garg, A. D. (2009) Efficient in Silico Designing of Oligonucleotides for Artificial Gene Synthesis. *Nat. Protoc.*, DOI: 10.1038/nprot.2009.15.
(48) Paladin, L., Schaeffer, M., Gaudet, P., Zahn-Zabal, M., Michel, P.-A., Piovesan, D., Tosatto, S. C. E., and Bairoch, A. (2020) The Feature-Viewer: A Visualization Tool for Positional Annotations on a Sequence. *Bioinformatics 36*, 3244−3245.
(49) Zahn-Zabal, M., Michel, P.-A., Gateau, A., Nikitin, F., Schaeffer, M., Audot, E., Gaudet, P., Duek, P. D., Teixeira, D., Rech de Laval, V., Samarasinghe, K., Bairoch, A., and Lane, L. (2019) The NeXtProt Knowledgebase in 2020: Data, Tools and Usability Improvements. *Nucleic Acids Res. 48*, D328−D334.
(50) SantaLucia, J. (1998) A Unified View of Polymer, Dumbbell, and Oligonucleotide DNA Nearest-Neighbor Thermodynamics. *Proc. Natl. Acad. Sci. U. S. A. 95*, 1460−1465.
(51) Nakamura, Y., Gojobori, T., and Ikemura, T. (2000) Codon Usage Tabulated from International DNA Sequence Databases: Status for the Year 2000. *Nucleic Acids Res. 28*, 292.
(52) Williams, E. M., Copp, J. N., and Ackerley, D. F. (2014) Site-Saturation Mutagenesis by Overlap Extension PCR. In *Directed Evolution Library Creation: Methods and Protocols* (Gillam, E. M. J., Copp, J. N., and Ackerley, D., Eds.), pp 83−101, Springer New York, New York, NY.